

現場 OS Lite 開発進捗報告書

チーム共有用 / 2026 年 6 月 21 日 時点

■ 現在地（サマリ）

Phase 1.0「基盤構築」がほぼ完了。 アプリの土台（サーバー接続・開発/本番の環境切替・画面の骨組み）を実装し、ソースコードを GitHub に保存、自動テスト（CI）も全項目グリーンを確認しました。残るは **iOS 実機での起動確認のみ**。これが済めば Phase 1.0 完了 → 最初の機能（ログイン）開発に進みます。

Ⅰ プロダクト概要

プロダクト 現場 OS Lite（仮称） — 中小建設企業の現場業務をスマホで支援するアプリ
対象ユーザー 建設現場の職人・監督
対応端末 iOS / Android（モバイル中心。Web は当面对象外）
開発規模 全 10 フェーズのロードマップ。現在は最初の土台（Phase 1.0）を構築中

Ⅱ 全体ロードマップ（Phase 1 → ローンチ）

Phase	内容	状態
1. 基盤＋初期機能	アプリの土台＋ログイン・現場一覧・写真管理	進行中
1.0 基盤構築	サーバー接続・dev/prod 切替・主要ライブラリ・CI・接続確認画面	進行中
1.1 認証/ログイン	ログイン画面・権限管理（会社単位）	未着手
1.2 現場一覧	自社の現場の一覧表示	未着手
1.3 写真管理	現場写真の撮影・保存・アップロード	未着手
2. 日報機能	日報の作成・一覧・編集	未着手
3. 現場管理の拡充	現場の登録/編集/状態管理・メンバー割当	未着手
4. 写真の本格化	オフライン同期・アルバム/タグ・位置情報	未着手
5. 通知・連携	プッシュ通知・現場内コミュニケーション	未着手
6. 帳票・出力	報告書/写真台帳の PDF 出力・共有	未着手
7. 組織・権限管理	招待フロー・ロール管理・管理画面	未着手
8. 品質強化	テスト網羅・クラッシュ監視・パフォーマンス	未着手
9. ベータ検証	TestFlight / Google Play 内部テスト配信	未着手
10. ストア公開	App Store / Google Play 申請・本番リリース	未着手

凡例： 完了 完了 進行中 進行中 未着手 未着手

Phase 1.0 「基盤構築」 — 完了した工程

土台づくりとして、以下を実装・検証しました。コードは自分の PC だけでなく、GitHub 上のまっさらな環境 (CI) でも正常動作することを確認済みです。

工程	状態
Supabase (サーバー) の開発用・本番用 2 環境を作成・接続鍵を取得	完了
環境別設定 (dev / prod の切替、接続情報の安全管理)	完了
主要ライブラリの選定・導入 (状態管理・画面遷移・モデル生成 等)	完了
アプリ構成 (feature-first アーキテクチャ) の骨組みを整備	完了
「サーバー接続 OK」確認画面の実装	完了
静的解析 (コード品質チェック) エラー 0	完了
自動テスト 6 件すべて成功	完了
ソースコードを GitHub に保存 (コミット & プッシュ)	完了
CI (GitHub Actions) の構築 — 保存のたびに自動でチェック	完了
CI をグリーン化 (Analyze+Test が自動で通る状態に)	完了
開発環境の不具合 (日本語フォルダ名問題) を解消	完了

Phase 1.0 完了までに残っている作業

残作業	状態
iOS のビルド設定 (pod install、Xcode の環境構成・スキーム作成)	未着手
iOS 実機/シミュレータで「接続 OK」を目視確認	未着手
(Android の実機起動確認は方針により後続フェーズへ)	後回し

※ iOS を優先し、Android は後続フェーズで対応する方針です。そのため現時点では CI の自動チェックも「コード解析+テスト」に絞っています (Android ビルド検証は着手時に追加)。

次の工程

STEP 1 iOS のビルド設定を行う (pod install → Xcode 構成 → スキーム作成)

STEP 2 iOS で起動し「環境: dev/接続: OK」を確認 → **これで Phase 1.0 完了**

STEP 3 Phase 1.1 「認証/ログイン」の開発に着手

技術・インフラ構成

項目	内容
フレームワーク	Flutter 3.44.2 (iOS / Android を 1 つのコードで開発)
バックエンド	Supabase — 開発用 genba-os-dev / 本番用 genba-os-prod の 2 環境
コード管理	GitHub リポジトリ shukura19960114-wq/genba-os-lite
自動チェック	GitHub Actions (CI): コード解析+テストを自動実行 — 現在グリーン
主要ライブラリ	supabase_flutter / flutter_riverpod / go_router / freezed ほか

補足: Supabase と GitHub の役割の違い

サービス	役割
Supabase	アプリのバックエンド。データの保存・ユーザー認証を担当。開発用と本番用で分離
GitHub	アプリのソースコードの保管庫+自動チェック (CI)。プログラムそのものを管理

※ 同じ 1 つのアプリ「現場 OS Lite」を、この 2 つのサービスが役割分担して支えています (別プロジェクトではありません)。